Green University of Bangladesh

Department of Computer Science and Engineering (CSE)
Semester: (Summer, Year: 2026), B.Sc. in CSE (Day)

TREASURE HUNT GAME

Course Title: Data Structure

Course Code: CSE 206

Section: 252_D3

Students Details

Name	ID
Mehedi Hasan Auntik	252002015

Submission Date: 23-08-2026

Course Teacher's Name: Md. Sabbir Hosen Mamun

Marks: _____ Signature: _____

Comments: _____ Date: _____

Contents

Chapter 1

Introduction

1.1 Overview

The **Treasure Hunt Game** is a menu-driven, console-based application developed in the C programming language as a laboratory project for the course *Data Structure (CSE 206)*. The primary purpose of this project is to demonstrate the practical implementation of fundamental data structures and algorithms within the context of an interactive game. In this game, a player navigates through an 8×8 grid-based board in search of a hidden treasure, while avoiding walls that can be dynamically inserted or removed. Alongside the core gameplay, the system integrates an inventory management module, a player leaderboard, an automatic path-finding solver, and a player record-keeping module, each of which is built using a distinct data structure. This makes the project a comprehensive showcase of arrays, linked lists, sorting, searching, graph traversal, and binary search trees, all combined within a single, cohesive application.

1.2 Motivation

Data structures form the backbone of efficient software design, yet learning them purely through isolated theoretical exercises can often feel abstract and disconnected from real-world problem solving. The motivation behind this project was to bridge that gap by embedding multiple data structure concepts into a single, engaging, and interactive application. By designing a treasure hunt game, the objective was to make the learning process more intuitive: array manipulation becomes the game board, linked lists become the player's inventory, graph traversal becomes the automatic solver, and binary search trees become the player record system. This approach reinforces conceptual understanding while simultaneously building practical programming and software design skills.

1.3 Problem Definition

1.3.1 Problem Statement

Given an $N \times N$ grid representing a game board, a player positioned at a starting cell, and a treasure located at a target cell, the problem is to design a system that allows the player to:

1. Navigate the board while dynamically avoiding obstacles (walls).
2. Manage a collection of collected items using efficient insertion and deletion operations.

3. Maintain and query a leaderboard of players ranked by score.
4. Determine, algorithmically, whether the treasure is reachable from the player's current position and compute the shortest path to it.
5. Store and retrieve player records efficiently based on their score.

Each of these sub-problems maps naturally onto a specific data structure, and the overall challenge is to integrate all of them into a single consistent and functional program.

1.3.2 Complex Engineering Problem

This project reflects the attributes of a Complex Engineering Problem (CEP) as it requires:

- **Depth of knowledge:** Application of multiple data structures (arrays, linked lists, trees, and graphs) and algorithms (searching, sorting, and traversal) learned throughout the course.
- **Range of conflicting requirements:** Balancing memory efficiency (dynamic allocation for linked lists and trees) against speed of access (array-based leaderboard for direct indexing).
- **Depth of analysis required:** Selecting the appropriate data structure for each sub-module based on the nature of the operations required (e.g., BFS for shortest-path computation instead of a simple linear scan).
- **Familiarity of issues:** While each data structure is individually well known, integrating them into a single working system with a common interface is a non-trivial engineering task.

1.4 Design Goals/Objectives

The key objectives of this project are outlined below:

- To implement a 2D array-based game board supporting insertion and deletion of obstacles (walls).
- To implement Linear Search and Binary Search algorithms for locating players on the leaderboard.
- To implement the Bubble Sort algorithm to rank players by score.
- To implement a singly linked list for managing the player's inventory, supporting insertion, deletion, and traversal.
- To implement Breadth-First Search (BFS) as a graph traversal technique to automatically determine the shortest path to the treasure.
- To implement a Binary Search Tree (BST) for efficiently storing and retrieving player records by score.
- To combine all of the above modules into a single, user-friendly, menu-driven console application.

1.5 Application

Although designed as an educational game, the underlying concepts demonstrated in this project have wide-ranging real-world applications:

- **2D Arrays:** Used in image processing, spreadsheet applications, and grid-based simulations such as maze solvers and cellular automata.
- **Linked Lists:** Used in dynamic memory management, undo/redo functionality in software, and implementation of other data structures such as stacks and queues.
- **Sorting and Searching:** Fundamental to database indexing, search engines, and any system that needs to rank or retrieve records efficiently.
- **Graph Traversal (BFS):** Used in GPS navigation systems, network routing protocols, and social network friend-suggestion algorithms.
- **Binary Search Trees:** Used in database indexing systems, file systems, and any application requiring fast insertion, deletion, and lookup of ordered data.

Chapter 2

Design/Development

2.1 Introduction

This chapter presents the design and implementation details of the Treasure Hunt Game. It describes the overall project structure, the data structures used for each functional module, and the algorithms implemented to achieve the desired functionality. The system was developed entirely in the C programming language using standard libraries such as `stdio.h`, `stdlib.h`, and `string.h`.

2.2 Project Details

The project is organized into six major functional modules, each corresponding to a specific data structure or algorithmic concept taught in the course:

1. **Board Module** – Implemented using a 2D character array to represent the game grid.
2. **Inventory Module** – Implemented using a singly linked list.
3. **Leaderboard Module** – Implemented using a 1D array of structures, with Bubble Sort, Linear Search, and Binary Search.
4. **Auto-Solver Module** – Implemented using Breadth-First Search (BFS) graph traversal.
5. **Player Record Module** – Implemented using a Binary Search Tree (BST).
6. **Menu-Driven Interface** – A `switch-case` based main loop that connects all modules together.

2.2.1 Structure

The core data structures used in the project are defined as follows:

Listing 2.1: Board and Global Variables

```
#define SIZE 8
#define MAX_PLAYERS 20

char board[SIZE][SIZE];
int playerRow, playerCol;
int treasureRow, treasureCol;
```

Listing 2.2: Linked List Node for Inventory

```
typedef struct Node {
    char item[20];
    struct Node* next;
} Node;
Node* inventory = NULL;
```

Listing 2.3: Binary Search Tree Node for Player Records

```
typedef struct TreeNode {
    char name[20];
    int score;
    struct TreeNode* left;
    struct TreeNode* right;
} TreeNode;
TreeNode* root = NULL;
```

Listing 2.4: Leaderboard Array Structure

```
typedef struct {
    char name[20];
    int score;
} Player;
Player leaderboard[MAX_PLAYERS];
int playerCount = 0;
```

2.3 Implementation

The implementation of each module is briefly described below.

Board Initialization and Display: The `initBoard()` function fills the 2D array with empty cells (`'.'`), places the player (`'P'`) at the starting position, and places the treasure (`'T'`) at the bottom-right corner. The `printBoard()` function traverses the 2D array and prints its current state to the console.

Wall Insertion and Deletion: The `insertWall()` and `deleteWall()` functions perform insertion and deletion operations directly on the 2D array by validating the given coordinates before modifying the corresponding cell.

Player Movement: The `movePlayer()` function updates the player's coordinates based on directional input (`w`, `a`, `s`, `d`), checks for board boundaries and wall collisions, and detects whether the treasure has been reached.

Inventory Management: The `insertItem()` function performs head insertion into the linked list, `deleteItem()` traverses the list to locate and remove a node while correctly re-linking the surrounding nodes, and `traverseInventory()` walks through the list to display all collected items.

Leaderboard Operations: The `addPlayer()` function appends a new player to the array. The `bubbleSortByScore()` function sorts the leaderboard in descending order of score, while `sortByNameForBinarySearch()` sorts it alphabetically to enable binary search.

Auto-Solver: The `bfsSolve()` function uses a queue-based Breadth-First Search to explore the board level by level from the player's position, determining the shortest path length to the treasure while avoiding walls.

Player Records: The `insertBST()`, `searchBST()`, and `inorderBST()` functions implement recursive insertion, searching, and in-order traversal on the Binary Search Tree, respectively.

2.4 Algorithms

Algorithm 1: Bubble Sort (Leaderboard by Score)

1. For i from 0 to $n - 2$:
2. For j from 0 to $n - i - 2$:
3. If $score[j] < score[j + 1]$, swap the two elements.
4. Repeat until the array is fully sorted in descending order.

Algorithm 2: Binary Search (Leaderboard by Name)

1. Set $low = 0$ and $high = n - 1$.
2. While $low \leq high$:
3. Compute $mid = (low + high)/2$.
4. If $name[mid]$ equals the target, return mid.
5. Else if $name[mid] < target$, set $low = mid + 1$.
6. Else set $high = mid - 1$.
7. If not found, return -1.

Algorithm 3: Breadth-First Search (Auto-Solver)

1. Initialize a queue and enqueue the player's starting cell with distance 0.
2. Mark the starting cell as visited.
3. While the queue is not empty:
4. Dequeue the front cell.
5. If it is the treasure cell, return the recorded distance.
6. Otherwise, enqueue all valid, unvisited, non-wall neighboring cells (up, down, left, right) with distance +1.
7. If the queue becomes empty without reaching the treasure, report that it is unreachable.

Algorithm 4: Binary Search Tree Insertion

1. If the current node is `NULL`, create a new node with the given name and score and return it.

2. If the new score is less than the current node's score, recursively insert into the left subtree.
3. Otherwise, recursively insert into the right subtree.
4. Return the (possibly updated) node.

Chapter 3

Performance Evaluation

3.1 Simulation Procedure

The project was developed and tested under the following environment:

- **Programming Language:** C (C99 standard)
- **Compiler:** GCC (GNU Compiler Collection)
- **IDE/Editor:** Code::Blocks / VS Code
- **Operating System:** Windows 10/11
- **Testing Method:** Manual black-box testing was performed by executing each menu option with a range of valid and invalid inputs, and observing the program's output for correctness.

Each module was tested independently before being integrated into the combined menu-driven system, ensuring that errors could be isolated and corrected early in the development process.

3.2 Results Analysis

3.2.1 Board and Wall Operations Result

The board initialization, player movement, and wall insertion/deletion functions were tested by adding walls at various coordinates and confirming that the player could no longer move through those cells. Boundary checks were also verified by attempting to move the player outside the 8×8 grid.

3.2.2 Search Player Result

Both Linear Search and Binary Search were tested against the leaderboard array. Linear Search correctly located players regardless of array order, while Binary Search was verified to function correctly only after the array was sorted alphabetically, confirming the precondition required for the algorithm.

3.2.3 Inventory and BST Result

The linked list-based inventory was tested by inserting and deleting multiple items and verifying correct traversal output. The Binary Search Tree was tested by inserting several player records with varying scores and confirming that the in-order traversal produced a correctly sorted sequence by score.

3.3 Results Overall Discussion

The testing process confirmed that all implemented modules function correctly both in isolation and when integrated into the complete system. The Bubble Sort and Binary Search Tree operations produced consistent, correctly ordered results. The BFS auto-solver accurately computed the shortest path length in all tested wall configurations, and correctly reported when the treasure was unreachable due to a fully blocked path.

3.3.1 Complex Engineering Problem Discussion

The project successfully addresses the complex engineering problem outlined in Chapter 1 by demonstrating the appropriate selection and integration of multiple data structures based on their respective strengths: arrays for fixed-size, index-based access (board and leaderboard); linked lists for dynamic, frequently changing collections (inventory); trees for hierarchical, ordered data (player records); and graph traversal for pathfinding (auto-solver). This selective application of data structures based on problem requirements reflects sound engineering judgment.

Chapter 4

Conclusion

4.1 Discussion

The Treasure Hunt Game successfully demonstrates the practical application of core data structure concepts, including arrays, linked lists, sorting algorithms, searching algorithms, graph traversal, and binary search trees, all integrated within a single interactive console application. The project achieved its design goals by providing a functional, menu-driven system in which each module operates correctly both independently and as part of the larger program.

4.2 Limitations

Despite meeting its objectives, the project has certain limitations:

- The game board size is fixed at compile time (8×8) and cannot be resized dynamically.
- The Binary Search Tree is not self-balancing, so in the worst case (e.g., inserting strictly increasing scores) it may degrade to a linked-list-like structure with $O(n)$ operations.
- Player and inventory data are not persisted to a file, meaning all progress is lost when the program terminates.
- The console-based interface lacks a graphical representation of the board, which limits user experience compared to a graphical game.

4.3 Scope of Future Work

The following enhancements are proposed for future development:

- Implement file handling to save and load player records, inventory, and leaderboard data between sessions.
- Replace the standard Binary Search Tree with a self-balancing tree (AVL Tree) to guarantee $O(\log n)$ performance in all cases.
- Develop a graphical user interface (GUI) using a library such as SDL or a framework such as Qt to enhance the visual experience.
- Extend the BFS auto-solver into an interactive hint system that highlights the next optimal move for the player.

- Introduce multiple difficulty levels with dynamically generated board sizes and wall configurations.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

2. Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.

3. GeeksforGeeks. (n.d.). *Data Structures*. Retrieved from https://www.geeksforgeeks.org/data-structures/

4. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Project Links

GitHub Repository: https://github.com/auntik75-hash/Treasure$_H$*unt*

Project Demonstration Video: https://youtu.be/SAIBk5faZbQ